

Iterables - Simple Hinglish Summary

Source: <https://javascript.info/iterable>

Main baat

Iterable objects arrays ka generalization hain. Ye aisa concept hai jisse **koi bhi object** `for..of` loop me use ho sakta hai.

Arrays iterable hain, lekin aur bhi built-in objects iterable hain. Jaise **strings** bhi iterable hain.

Object array na ho lekin kisi cheez ka collection (list, set) ho, to `for..of` uspar loop chalane ka achha syntax hai.

1. `Symbol.iterator`

Iterable samajhne ka sabse aasan tarika: **khud ek banao**.

Maan lo ek `range` object hai jo numbers ka interval dikhata hai:

```
javascript

let range = {
  from: 1,
  to: 5
};

// hum chahte hain ki for..of chale:
// for(let num of range) ... num=1,2,3,4,5
```

`range` ko iterable banane ke liye object me `Symbol.iterator` naam ka method (special built-in symbol) add karna padta hai.

Kaise kaam karta hai:

- `for..of` shuru hote hi ye method **ek baar** call karta hai (na mile to error). Ye method ek **iterator** return karna chahiye, yaani ek object jisme `next` method ho.
- Uske baad `for..of` **sirf usi returned object** ke saath kaam karta hai.
- Agli value chahiye to `for..of` us object par `next()` call karta hai.
- `next()` ka result `{done: Boolean, value: any}` form me hona chahiye. `done=true` ka matlab loop khatam. Warna `value` agli value hai.

`range` ka poora implementation:

```
javascript
```

```

let range = {
  from: 1,
  to: 5
};

// 1. for..of shuru me ise call karta hai
range[Symbol.iterator] = function() {

  // ...ye iterator object return karta hai:
  // 2. Aage for..of sirf is iterator object se kaam karta hai, usse agli value
  return {
    current: this.from,
    last: this.to,

    // 3. next() har iteration par for..of loop call karta hai
    next() {
      // 4. ye {done:..., value :...} object return kare
      if (this.current <= this.last) {
        return { done: false, value: this.current++ };
      } else {
        return { done: true };
      }
    }
  };
};

// ab chalta hai!
for (let num of range) {
  alert(num); // 1, phir 2, 3, 4, 5
}

```

Iterables ki core khoobi: separation of concerns (zimmedari alag).

- `range` ke paas khud `next()` method nahi hai.
- Iterator naam ka alag object `range[Symbol.iterator]()` call se banta hai, aur uska `next()` iteration ki values generate karta hai.

Yaani iterator object us object se alag hai jis par iterate ho raha hai.

Simple version: technically dono ko merge karke `range` ko hi iterator bana sakte hain:

javascript

```

let range = {
  from: 1,
  to: 5,

  [Symbol.iterator]() {
    this.current = this.from;
    return this;
  },

  next() {
    if (this.current <= this.to) {
      return { done: false, value: this.current++ };
    } else {
      return { done: true };
    }
  }
};

for (let num of range) {
  alert(num); // 1, phir 2, 3, 4, 5
}

```

Ab `range[Symbol.iterator]()` `range` ko hi return karta hai: usme `next()` hai aur wo `this.current` me iteration ka progress yaad rakhta hai. Chhota hai, aur kabhi kabhi theek bhi.

Nuksan: ab object par `do for..of` loops **ek saath** nahi chal sakte, kyunki dono iteration state share karenge (iterator ek hi hai, object khud). Lekin do parallel for-of bahut rare hain.

Infinite iterators: ye bhi possible hain. `range.to = Infinity` karo to `range` infinite ho jata hai. Ya pseudorandom numbers ka infinite sequence banane wala iterable bhi ban sakta hai. `next` par koi limit nahi hai, wo aur values deta rahe to normal hai. Aise iterable par `for..of` loop kabhi khatam nahi hoga, lekin `break` se hamesha rok sakte hain.

2. String iterable hai

Arrays aur strings sabse zyada use hone wale built-in iterables hain. String par `for..of` uske **characters** par chalta hai:

```

javascript

for (let char of "test") {
  // 4 baar chalta hai: har character ke liye ek baar
  alert(char); // t, phir e, phir s, phir t
}

```

Aur ye **surrogate pairs** ke saath bhi sahi kaam karta hai:

```
javascript

let str = 'x😂';
for (let char of str) {
    alert( char ); // x, phir 😂
}
```

3. Iterator ko explicitly call karna

Gehri samajh ke liye dekhte hain ki iterator ko direct kaise use karein. String par bilkul `for...of` jaise hi, lekin direct calls se:

```
javascript

let str = "Hello";

// ye is ke barabar hai:
// for (let char of str) alert(char);

let iterator = str[Symbol.iterator]();

while (true) {
    let result = iterator.next();
    if (result.done) break;
    alert(result.value); // characters ek ek karke
}
```

Iski zarurat kam padti hai, lekin `for...of` se **zyada control** milta hai. Jaise iteration ko baant sakte ho: thoda iterate karo, ruko, kuch aur karo, aur phir baad me dobara shuru karo.

4. Iterables aur array-likes

Do official terms jo dekhne me milte-julte hain lekin bahut alag hain:

- **Iterables:** wo objects jo `Symbol.iterator` method implement karte hain (upar wale tarike se).
- **Array-likes:** wo objects jinme **indexes** aur `length` hote hain, yaani array jaise dikhte hain.

Browser ya kisi aur environment me hum iterables, array-likes ya dono se mil sakte hain.

Jaise **strings dono hain:** iterable (`for...of` chalta hai) aur array-like (numeric indexes aur `length` hai).

Lekin iterable array-like ho, ye zaruri nahi. Aur array-like iterable ho, ye bhi zaruri nahi.

Jaise upar ka `range` **iterable hai lekin array-like nahi** (indexed properties aur `length` nahi hai).

Ye object **array-like hai lekin iterable nahi**:

javascript

```
let arrayLike = { // indexes aur length hai => array-like
  0: "Hello",
  1: "World",
  length: 2
};

// Error (Symbol.iterator nahi hai)
for (let item of arrayLike) {}
```

Iterables aur array-likes dono aam taur par **arrays nahi** hote, unme `push`, `pop` jaise methods nahi hote. Agar aise object ko array ki tarah use karna ho (jaise `range` par array methods), to?

5. `Array.from`

Ek universal method `Array.from` hai jo kisi bhi iterable ya array-like value se **asli Array** banata hai. Phir us par array methods chala sakte hain.

javascript

```
let arrayLike = {
  0: "Hello",
  1: "World",
  length: 2
};

let arr = Array.from(arrayLike); // (*)
alert(arr.pop()); // World (method chalta hai)
```

`Array.from` line `(*)` par object ko dekhta hai ki wo iterable hai ya array-like, phir naya array banata hai aur saare items copy karta hai.

Iterable ke saath bhi wahi hota hai:

javascript

```
// maan lo range upar wale example se hai
let arr = Array.from(range);
alert(arr); // 1,2,3,4,5
```

Array.from ka poora syntax optional "mapping" function bhi deta hai:

```
javascript

Array.from(obj[, mapFn, thisArg])
```

Optional dusra argument **mapFn** ek function hai jo har element par array me jodne se pehle lagta hai, aur **thisArg** uska **this** set karta hai.

```
javascript

// har number ka square
let arr = Array.from(range, num => num * num);

alert(arr); // 1,4,9,16,25
```

String ko characters ke array me badalna:

```
javascript

let str = 'x😂';

// str ko characters ke array me todo
let chars = Array.from(str);

alert(chars[0]);    // x
alert(chars[1]);    // 😂
alert(chars.length); // 2
```

str.split ke ulta ye string ke iterable hone par depend karta hai, isliye **for..of** ki tarah **surrogate pairs sahi handle karta hai**.

Technically ye is jaisa hai:

```
javascript
```

```
let chars = []; // Array.from andar se yahi loop chalata hai
for (let char of str) {
  chars.push(char);
}
```

...lekin chhota hai.

Isse **surrogate-aware** `slice` bhi bana sakte hain:

```
javascript

function slice(str, start, end) {
  return Array.from(str).slice(start, end).join('');
}

let str = 'X😄𐤎';

alert( slice(str, 1, 3) ); // 😄𐤎

// native method surrogate pairs support nahi karta
alert( str.slice(1, 3) ); // kachra (alag surrogate pairs ke do tukde)
```

Summary

Jo objects `for...of` me use ho sakte hain unhe **iterable** kehte hain.

- Technically iterables ko `Symbol.iterator` naam ka method implement karna padta hai.
 - `obj[Symbol.iterator]()` ke result ko **iterator** kehte hain. Wo aage ki iteration process handle karta hai.
 - Iterator me `next()` method hona chahiye jo `{done: Boolean, value: any}` return kare. `done:true` iteration ka end dikhata hai, warna `value` agli value hai.
- `Symbol.iterator` method `for...of` automatically call karta hai, lekin hum direct bhi kar sakte hain.
- Strings ya arrays jaise built-in iterables bhi `Symbol.iterator` implement karte hain.
- String iterator **surrogate pairs** ke bare me jaanta hai.

Indexed properties aur `length` wale objects ko **array-like** kehte hain. Inme aur properties aur methods bhi ho sakte hain, lekin arrays ke built-in methods nahi hote.

Specification dekho to zyadatar built-in methods "asli arrays" ki jagah iterables ya array-likes ke saath kaam karte hain, kyunki wo zyada abstract hai.

`Array.from(obj[, mapFn, thisArg])` iterable ya array-like `obj` se asli `Array` banata hai, aur phir array methods use kar sakte hain. Optional `mapFn` aur `thisArg` har item par function lagane dete hain.

Quick Summary

Baat	Yaad rakho
Iterable	<code>Symbol.iterator</code> method wala object, <code>for...of</code> me chalta hai
Iterator	<code>next()</code> wala object, <code>{done, value}</code> return karta hai
<code>done: true</code>	Iteration khatam
Built-in iterables	Arrays, strings, Map, Set
Array-like	Indexes + <code>length</code> (jaise <code>{0: "a", length: 1}</code>)
Iterable vs array-like	Ek doosre se alag concepts (string dono hai)
Array-like par <code>for...of</code>	Error (agar iterable nahi hai)
Asli array banana	<code>Array.from(obj)</code>
Mapping ke saath	<code>Array.from(obj, x => x * x)</code>
String → characters	<code>Array.from(str)</code> (surrogate pairs sahi)
Infinite iterable	<code>for...of</code> me <code>break</code> se rokna padta hai

Yaad rakho: Iterable = `for...of` chalne layak. Array-like = index aur length wala. Dono ko asli array banane ka tarika `Array.from`.